

AIML320 Assignment 3 — Joshua Pinpin (ID: 300662880)

Part 1: Job Shop Scheduling

The instance has 3 jobs and 2 machines:

Job	Arrival	Op 1 (machine, time)	Op 2 (machine, time)
J1	0	O11 (M1, 50)	O12 (M2, 25)
J2	10	O21 (M2, 30)	O22 (M1, 35)
J3	20	O31 (M1, 40)	O32 (M2, 20)

Every schedule must satisfy three constraints: an operation O_{j1} cannot start before job J_j arrives (arrival), O_{j2} cannot start before O_{j1} has finished (precedence), and each machine processes at most one operation at a time (resource).

Q1. Earliest starting times (FCFS sequence)

The given sequence is:

Process(O11, M1, t_1) → Process(O21, M2, t_2) → Process(O31, M1, t_3) → Process(O12, M2, t_4) → Process(O22, M1, t_5) → Process(O32, M2, t_6)

This fixes the order of operations on each machine: M1 processes O11, then O31, then O22, and M2 processes O21, then O12, then O32. Following the hint, the earliest start time of each action is

- **ready time** of the operation = the job's arrival time (for O_{j1}), or the finish time of O_{j1} (for O_{j2}),
- **idle time** of the machine = the finish time of the previous operation on that machine (0 if none),
- **start** = max(ready time, idle time), and **finish** = start + processing time.

Working through the actions in order, and updating each machine's idle time after each step (both machines start idle at time 0):

#	Action	Proc time	Op ready time	Machine idle time	Start = max	Finish
1	O11 on M1	50	0 (J1 arrives)	M1: 0	$t_1 = 0$	50
2	O21 on M2	30	10 (J2 arrives)	M2: 0	$t_2 = 10$	40
3	O31 on M1	40	20 (J3 arrives)	M1: 50 (after O11)	$t_3 = 50$	90
4	O12 on M2	25	50 (O11 finishes)	M2: 40 (after O21)	$t_4 = 50$	75
5	O22 on M1	35	40 (O21 finishes)	M1: 90 (after O31)	$t_5 = 90$	125
6	O32 on M2	20	90 (O31 finishes)	M2: 75 (after O12)	$t_6 = 90$	110

Which constraint determines each start time:

1. **$t_1 = 0$** . J1 arrives at 0 and M1 is idle, so O11 starts immediately.
2. **$t_2 = 10$** . M2 is idle from time 0, but J2 only arrives at 10, so the arrival constraint is binding.

3. **t₃ = 50**. O31 is ready at 20 (J3's arrival), but M1 is busy with O11 until 50, so the resource constraint is binding.
4. **t₄ = 50**. M2 is free at 40 (after O21), but O12 cannot start until O11 finishes at 50, so the precedence constraint is binding.
5. **t₅ = 90**. O22 is ready at 40 (O21 has finished), but M1 is busy with O31 until 90, so the resource constraint is binding.
6. **t₆ = 90**. M2 is free at 75 (after O12), but O32 cannot start until O31 finishes at 90, so the precedence constraint is binding.

As a check, the start times satisfy $t_1 \leq t_2 \leq \dots \leq t_6$ ($0 \leq 10 \leq 50 \leq 50 \leq 90 \leq 90$), as required by the sorted sequence. The resulting schedule is:

Process(O11, M1, 0) → Process(O21, M2, 10) → Process(O31, M1, 50) → Process(O12, M2, 50) → Process(O22, M1, 90) → Process(O32, M2, 90)

Figure 1 shows this schedule as a Gantt chart. The waits identified above are visible as gaps on M2: it is idle at 0–10 until J2 arrives, and at 40–50 and 75–90 while O12 and O32 wait for their first operations to finish.

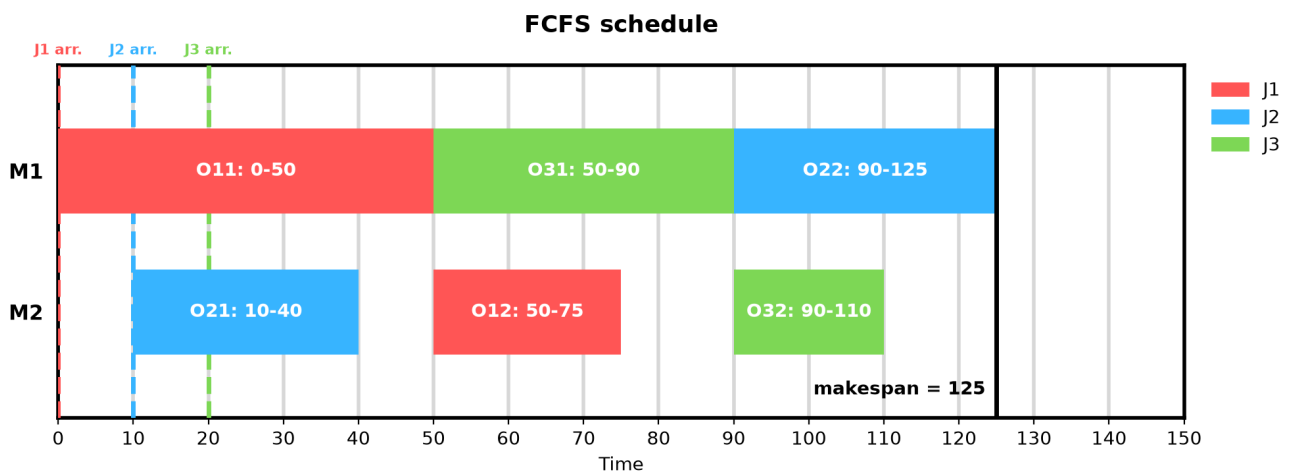


Figure 1: Gantt chart of the FCFS schedule from Q1 (makespan 125). Each bar is labelled with its operation and its start–finish times, and bars are coloured by job (key on the right). Dashed lines mark when each job arrives, the solid black line marks the makespan, and gaps between bars are machine idle time.

Q2. Completion times and makespan (FCFS)

The completion time of a job is the finish time of its last (second) operation, taken from the table in Q1:

Job	Last operation	Start	Proc time	Completion time
J1	O12	50	25	75
J2	O22	90	35	125
J3	O32	90	20	110

The makespan is the largest completion time over all jobs:

Makespan = max(75, 125, 110) = 125

J2 determines the makespan. Although J2 arrives second and its first operation O21 is finished by 40, its second operation O22 has to wait until 90 for M1, because the sequence puts O31 ahead of O22 on M1.

Q3. SPT schedule

A dispatching rule does not plan the whole schedule in advance. Instead, it simulates time moving forward and makes one decision at a time:

- Each machine has a **queue** of operations that are ready for it. An operation is ready when its job has arrived (for Oj1) or when Oj1 has finished (for Oj2).
- Whenever a machine is **idle** and its queue is **non-empty**, the rule chooses which queued operation to start next. The **Shortest Processing Time (SPT)** rule chooses the operation with the smallest processing time.
- A machine with an empty queue stays idle, even if an operation will become ready soon (non-delay dispatching).

The decision points are the times when a job arrives or an operation finishes, since these are the only times a queue or a machine's status can change. The trace below lists each decision point, with the processing times of queued operations in brackets.

Time t	Event(s)	M1 queue	M2 queue	Decision
0	J1 arrives → O11 ready	{O11(50)}	{}	M1 idle: starts O11, finishes at 50
10	J2 arrives → O21 ready	M1 busy	{O21(30)}	M2 idle: starts O21, finishes at 40
20	J3 arrives → O31 ready	{O31(40)} (M1 busy)	{}	M1 busy until 50: O31 waits
40	O21 finishes → O22 ready	{O31(40), O22(35)} (M1 busy)	{}	M2 idle but its queue is empty
50	O11 finishes → O12 ready	{O31(40), O22(35)}	{O12(25)}	M1: SPT picks O22 (35 < 40) , finishes at 85. M2: starts O12, finishes at 75
75	O12 finishes (J1 complete)	M1 busy	{}	M2 idle, queue empty
85	O22 finishes (J2 complete)	{O31(40)}	{}	M1 starts O31, finishes at 125
125	O31 finishes → O32 ready	{}	{O32(20)}	M2 starts O32, finishes at 145
145	O32 finishes (J3 complete)	{}	{}	All jobs done

The only real choice happens at **t = 50**. M1 becomes free, and two operations are waiting for it: O31 (ready since 20, 40 time units) and O22 (ready since 40, 35 time units). SPT picks O22 because it is shorter. FCFS would pick O31 because it has been waiting longer. Applying the same procedure with FCFS reproduces the Q1 schedule exactly, so this single decision is the only difference between the two solutions.

The final SPT solution, sorted by start time, is:

Process(O11, M1, 0) → Process(O21, M2, 10) → Process(O22, M1, 50) → Process(O12, M2, 50) → Process(O31, M1, 85) → Process(O32, M2, 125)

O22 and O12 both start at 50, so either order is valid. They are listed with M1 first, consistent with the tie at $t = 50$ in the Q1 sequence.

To double-check the hand calculations, I wrote a small optional script (`part1/gantt.py`). It checks both schedules against the arrival, precedence and resource constraints, and simulates non-delay dispatching with each rule. The simulation produces exactly the Q1 schedule when using FCFS and exactly the schedule above when using SPT. Figure 2 shows the SPT schedule.

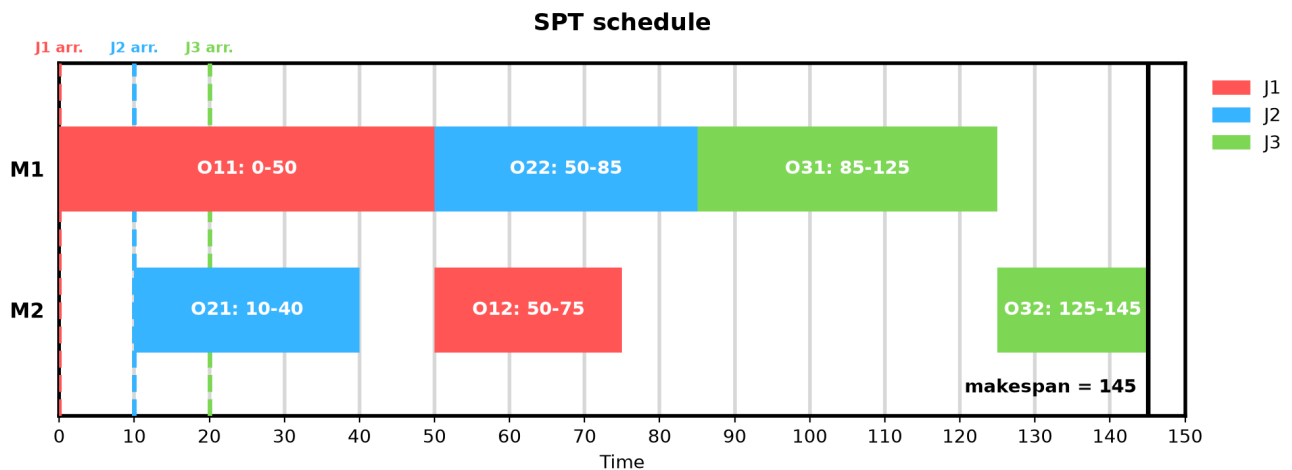


Figure 2: Gantt chart of the SPT schedule from Q3 (makespan 145), drawn on the same time axis as Figure 1. M2 is idle from 75 to 125 while it waits for O31 to finish on M1.

Q4. SPT completion times, makespan and comparison

As in Q2, each job's completion time is the finish time of its second operation:

Job	Last operation	Start	Proc time	Completion time
J1	O12	50	25	75
J2	O22	50	35	85
J3	O32	125	20	145

Makespan = $\max(75, 85, 145) = 145$

Comparison with the FCFS solution from Q1 and Q2:

	J1	J2	J3	Makespan
FCFS (Q1)	75	125	110	125
SPT (Q3)	75	85	145	145

In terms of makespan, the FCFS solution is better: 125 compared with 145, a difference of 20 time units.

SPT helps J2, which finishes 40 units earlier (85 instead of 125), but it delays J3 by 35 units (145 instead of 110). This happens because SPT runs O22 before O31 on M1, so O31 does not finish until 125. O32 cannot

start on M2 until then, and M2 sits idle from 75 to 125. J3 now determines the makespan, and it is 20 units worse than under FCFS.

Q5. Does a better solution imply a better rule?

No. FCFS producing the better solution on this instance does not mean FCFS is a better rule than SPT in general, for three reasons.

1. "Better" depends on the objective. Makespan is only one way to measure a schedule. The table below compares both solutions on two other common objectives: mean flowtime (the average time a job spends in the shop, completion time minus arrival time) and total completion time. The values were computed from the completion times in Q2 and Q4.

Rule	Completion (J1, J2, J3)	Flowtime (J1, J2, J3)	Mean flowtime	Total completion time	Makespan
FCFS	75, 125, 110	75, 115, 90	93.33	310	125
SPT	75, 85, 145	75, 75, 125	91.67	305	145

SPT is worse on makespan but better on both flowtime-based objectives, so even on this one instance neither rule dominates the other.

2. One small instance is not enough evidence. This instance has only 3 jobs and 2 machines, and the two solutions differ in a single decision (at $t = 50$). Dispatching rules are heuristics, and how well they perform varies a lot between instances. Small changes can flip or remove the difference. For example, if J3 arrived after $t = 50$, O31 would not be in M1's queue at that decision point, and both rules would produce the same schedule. To conclude that one rule is better, both rules should be tested on many instances (e.g. randomly generated instances or a standard benchmark set), and their average performance should be compared, ideally with a statistical significance test.

3. Each rule suits different goals. SPT is greedy and myopic. At $t = 50$ it chose O22 (35) over O31 (40) because O22 is shorter, without considering that J3 still had 60 units of work left (O31 + O32) while J2 had only 35. Delaying J3 left M2 idle from 75 to 125 and increased the makespan. A rule that considers remaining work, such as Most Work Remaining (MWKR), would have chosen O31 here and is generally better suited to makespan. SPT, by finishing short operations first, tends to reduce flowtime-type objectives, which is what happened here. So which rule is "better" depends on what the scheduler is trying to optimise.

Q6. (AIML420 only)

Q6 is required for AIML420 students only. This report is for AIML320, so Q6 was not attempted.

Part 2: Neural Networks

Part 2 compares a single perceptron that I implemented from scratch with a library multilayer perceptron (MLP). The code is in two Jupyter notebooks: `part2/perceptron.ipynb` (Tasks a and b) and `part2/mlp.ipynb` (Task c). Both notebooks were run from top to bottom, and their saved outputs contain every table and figure in this part. Accuracy, the fraction of test samples classified correctly, is the metric throughout.

Data

Each dataset has a separate train and test file, and the target column is `class` (0 or 1).

Dataset	Train rows	Test rows	Features	Class 1 share (train / test)
SeaSyn	100	40	<code>attrib1</code> , <code>attrib2</code> , <code>attrib3</code> (values between 0 and 10)	52.0% / 52.5%
RingSyn	453	300	<code>feature1</code> , <code>feature2</code> (values between about -1.2 and 1.1)	66.9% / 65.7%

There are two things to note about the data:

- **The SeaSyn test rows are copies of the training data.** The 40 rows of SeaSynTest are identical to the first 40 rows of SeaSynTrain. The SeaSyn test accuracy in Task a therefore measures how well the perceptron fits its own training data, not how well it generalises. The RingSyn train and test files share no rows.
- **RingSyn leans towards class 1.** A model that always predicts class 1 already gets **0.657** on the RingSyn test set. I use this majority-class baseline as the yardstick in Tasks b and c, because a useful model has to beat it clearly.

The handout calls the RingSyn features `x1` and `x2`, but the files name them `feature1` and `feature2`. The notebooks treat the last column of each file as the class and every other column as a feature, so they don't depend on the column names.

Perceptron implementation

The perceptron is the `Perceptron` class in `part2/perceptron.ipynb`, written with numpy only. It has three methods:

- `activation(z)` is the step function. It returns 1 if $z \geq 0$ and 0 otherwise.
- `predict(X)` computes the net input $w \cdot x + b$ for each row and passes it through `activation`.
- `fit(X, y)` trains the weights with the perceptron learning rule.

The notebook also has two small helpers. `load(name)` reads a CSV file into features, labels and feature names. `make_scaler(X_train)` returns a function that standardises data using the training set's mean and standard deviation.

Part	Choice	Reason
Activation	Step function: $\hat{y} = 1$ if $w \cdot x + b \geq 0$, otherwise $\hat{y} = 0$	This is the classic Rosenblatt perceptron, and its 0/1 output matches the dataset labels.
Bias	A separate bias term b , updated like a weight on a constant input of 1	Without it, the decision boundary would have to pass through the origin.
Initialisation	$w = 0$ and $b = 0$	Deterministic, so the results depend only on the shuffle order.
Learning rule	Online (one sample at a time): $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$	$y - \hat{y}$ is 0 when the prediction is correct, so the weights only change on mistakes. A false negative ($y = 1, \hat{y} = 0$) moves the boundary towards x , and a false positive moves it away.
Learning rate	$\eta = 1.0$	Because the weights start at zero, every update is a multiple of η . η therefore only scales w and b , and any positive η gives exactly the same predictions.
Epoch	One full pass over the training set, in a new random order each epoch	Shuffling stops the result depending on the order of rows in the file. The shuffle uses a fixed seed (42), so every run is reproducible.
Feature scaling	Each feature is standardised to mean 0 and standard deviation 1, using the training set's mean and std, and the same values are applied to the test set	SeaSyn features range from 0 to 10, while the bias input is always 1. Without scaling, the bias would learn much more slowly than the weights. Using only the training set's statistics stops test data leaking into training.

The experiment is the `epoch_sweep` function. For each number of epochs in the handout's list, it trains a fresh perceptron, prints the test accuracy, and keeps the model. It then calls `plot_boundaries`, which draws each model's decision boundary over the training data in a 3×4 grid. Every model uses the same seed, so it sees the same shuffle order. The model trained for E epochs is therefore exactly the first E epochs of the 200-epoch model, and each row of the results is a snapshot of one training run.

Task a: Perceptron on linearly separable data (SeaSyn)

Test accuracy on SeaSynTest after training on SeaSynTrain for each number of epochs:

Model (epoch)	Accuracy
Perceptron (1)	0.675
Perceptron (5)	0.700
Perceptron (10)	0.675
Perceptron (15)	0.550
Perceptron (20)	0.850
Perceptron (50)	0.550
Perceptron (60)	0.750
Perceptron (80)	0.750
Perceptron (100)	0.700
Perceptron (120)	0.775
Perceptron (150)	0.400
Perceptron (200)	0.775

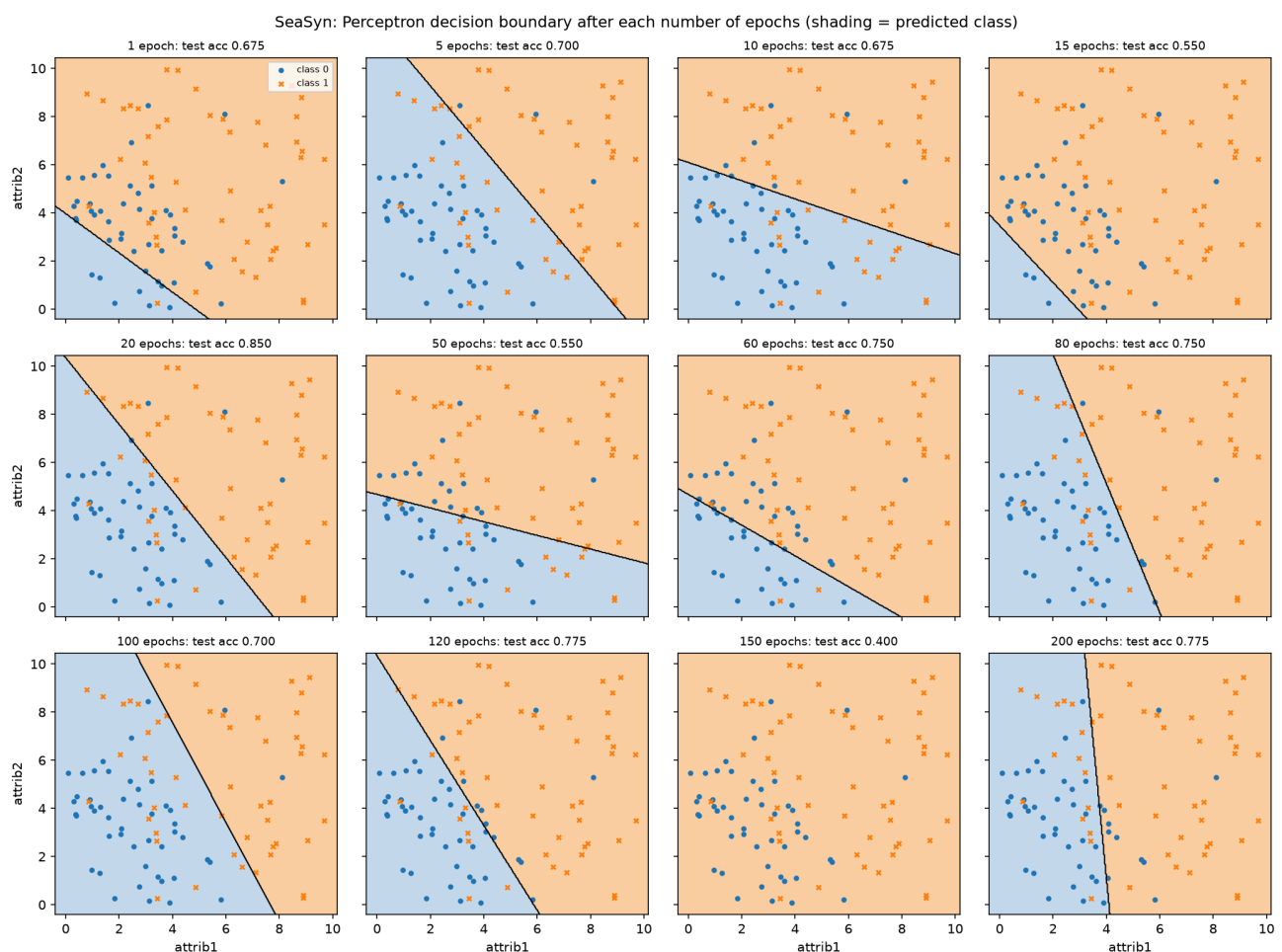


Figure 3: The decision boundary of each model in the table, drawn over the SeaSyn training data. Blue shading is predicted class 0 and orange is predicted class 1. SeaSyn has three features, so each panel is a 2D slice through *attrib1* and *attrib2*, with *attrib3* fixed at its training mean. A point's real prediction also depends on its own *attrib3* value, so a few points can be classified differently from the shading behind them.

How does increasing the number of epochs affect the model's accuracy?

On SeaSyn, training for more epochs does **not** make the perceptron more accurate. Test accuracy jumps up and down between rows with no upward trend. It is 0.850 after 20 epochs, then falls to 0.550 after 50 epochs and to 0.400 after 150 epochs. After 200 epochs it is 0.775, only 10 percentage points better than after 1 epoch (0.675) and worse than after 20 epochs.

The reason is that the perceptron **never converges** on this data. If it reached an epoch with no mistakes, its weights would stop changing, and every later row in the table would show the same accuracy. Instead, the accuracy and the boundary in Figure 3 are still changing between 150 and 200 epochs, so the perceptron is still making mistakes, and still updating, at the end of training. The Perceptron Convergence Theorem says that on linearly separable data the perceptron makes only a finite number of mistakes, after which it stops changing. Still making mistakes after 200 passes over 100 samples suggests that SeaSyn is **not linearly separable**, despite the task's title, and a separate check confirms it: a linear-programming feasibility test finds no weights w and bias b for which $w \cdot x + b$ puts every SeaSyn training point on its correct side. Accuracy never reaches 1.0 either, even though the 40 test rows are training rows. Figure 3 is consistent with this. The classes are roughly divided by a diagonal line in (attrib1, attrib2), but in every panel some points of each class sit on the wrong side of it. This looks like label noise: SeaSyn appears to follow the SEA concept (Street & Kim, 2001), which deliberately adds noise to about 10% of the class labels. The simple SEA-style rule "class 1 if attrib1 + attrib2 > 8" classifies 88% of the training rows correctly, which fits this: even a good line misses roughly one point in eight.

Because some training points are always misclassified, every epoch moves the boundary. Each update fixes the one point that was just misclassified, but it can move the line past many other points. The model returned after E epochs is wherever the last few updates left the line, not the best line seen during training. Stopping at a different epoch is like taking a random snapshot, which is why the accuracy depends far more on where training stops than on how long it ran. Figure 3 shows this directly:

- After 20 epochs (0.850) and 120 epochs (0.775), the line runs diagonally between the two classes, which is roughly the right split.
- After 10 and 50 epochs, the line is too flat and cuts through the class 1 points.
- After 150 epochs (0.400), the line has left the slice entirely, so the whole slice is predicted as class 1.

Two caveats apply. The test set has only 40 samples, so each mistake changes test accuracy by 2.5 percentage points. Also, because the test rows are copies of training rows, these numbers say little about how well the perceptron generalises to new data.

Task b: Perceptron on non-linearly separable data (RingSyn)

Task b reuses the Task a perceptron and `epoch_sweep` function without any changes. Only the data files are different.

Model (epoch)	Accuracy
Perceptron (1)	0.743
Perceptron (5)	0.697
Perceptron (10)	0.757
Perceptron (15)	0.727
Perceptron (20)	0.640
Perceptron (50)	0.747
Perceptron (60)	0.790
Perceptron (80)	0.730
Perceptron (100)	0.657
Perceptron (120)	0.773
Perceptron (150)	0.727
Perceptron (200)	0.747

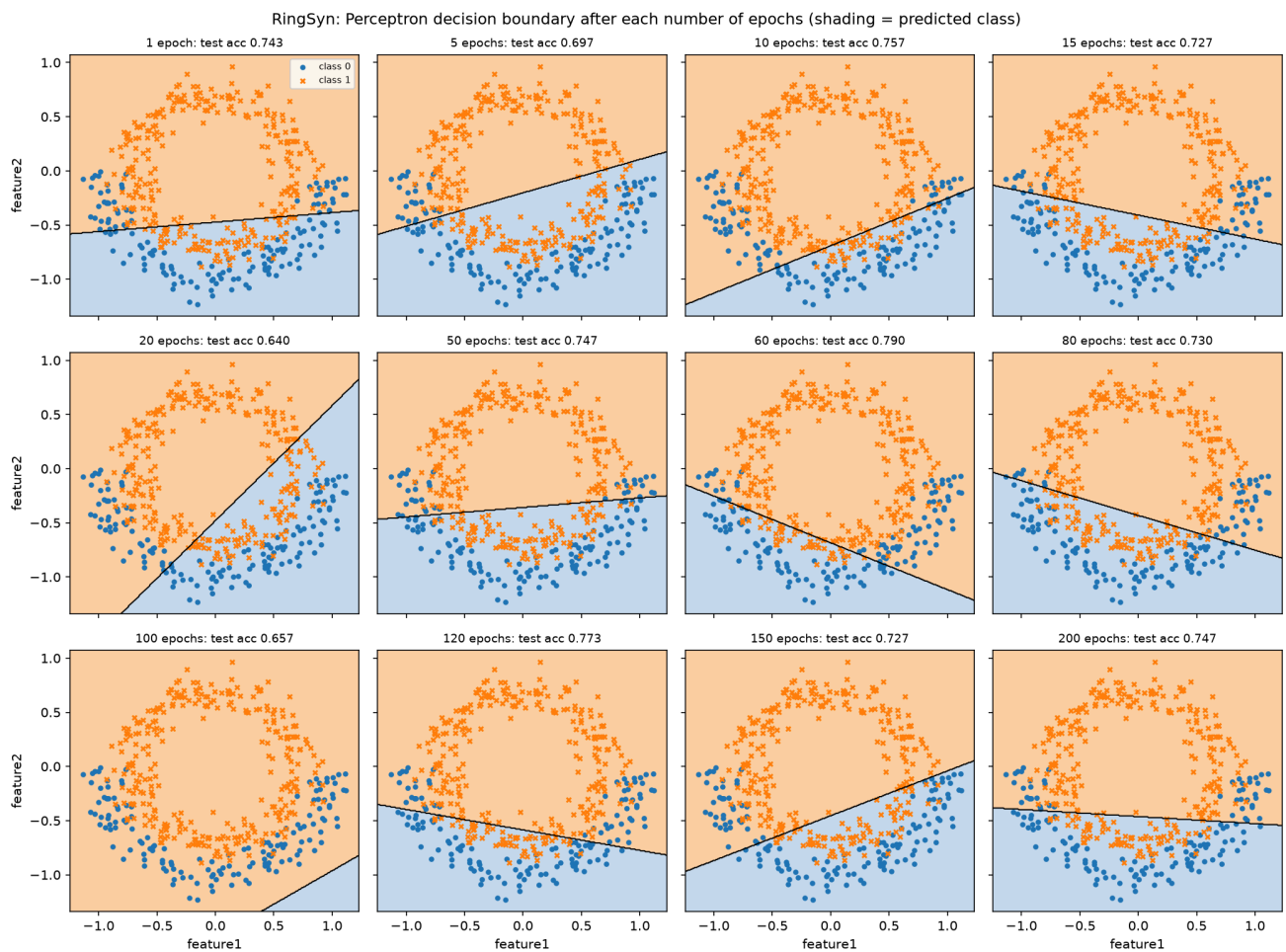


Figure 4: The decision boundary of each perceptron in the table, drawn over the RingSyn training data. RingSyn has only two features, so this is the full picture, not a slice.

How well does the perceptron perform on this dataset?

The perceptron performs **poorly** on RingSyn. Its test accuracy ranges from 0.640 to 0.790, and it is 0.747 after 200 epochs. The majority-class baseline gets 0.657, so the perceptron is at most 13 percentage

points better than always predicting class 1, and usually much less. The 20-epoch model (0.640) is worse than the baseline. The 100-epoch model scores exactly the baseline, 0.657: Figure 4 shows its line pushed into the bottom-right corner, so it predicts class 1 for every test point. As in Task a, more epochs do not help. The accuracy jumps between rows with no trend, and the boundary is still moving at 200 epochs, so the perceptron never converges.

The reason is the shape of the data, which Figure 4 makes clear. Class 1 (orange) forms a complete ring around the origin. Class 0 (blue) forms a wider half-ring outside it that only covers the lower half: every class 0 point has $\text{feature2} < 0$. A perceptron can only draw one straight line, and no straight line can separate a ring from the arc around it. The best it manages is what most panels show: a roughly horizontal line across the lower half, predicting class 1 above it. This gets the top half right, because it is all class 1, but it makes two kinds of mistake in the lower half:

- The bottom of the inner ring falls below the line and is predicted as class 0.
- The two ends of the outer half-ring curve up towards $\text{feature2} \approx 0$, so they stay above the line and are predicted as class 1.

Moving the line up fixes some of the second kind of mistake but makes more of the first, and moving it down does the opposite. The perceptron keeps swapping between these errors, so its line keeps tilting and moving (compare the 20, 60 and 100 epoch panels). Each row in the table is just a snapshot of that moving line.

The two datasets also fail in different ways. SeaSyn has a good linear boundary, and only a small number of noisy points stop the perceptron settling on it, so its best rows (0.850) are far above SeaSyn's majority baseline (0.525). RingSyn has no good linear boundary at all, so even the perceptron's best row (0.790) is only a modest improvement on guessing the majority class. This is a limit of the model, not of the training. A single perceptron can only represent a straight-line boundary, however long it is trained.

Task c: MLP on non-linearly separable data (RingSyn)

Setup. `part2/mlp.ipynb` uses scikit-learn's `MLPClassifier` with **one hidden layer of 8 neurons** and the **tanh** activation, trained with the default **adam** optimiser (`random_state=42`). As for the perceptron, both features are standardised using the training set's mean and standard deviation, with the same `make_scaler` helper. I chose 8 neurons, in the middle of the suggested 4–10, so the network has enough neurons to bend its boundary around the inner ring. I chose tanh because it is smooth, which suits a curved boundary. It uses the same RingSyn train and test files as Task b.

The notebook mirrors the perceptron notebook. Its `epoch_sweep` and `plot_boundaries` functions are the same as before, except that each model is `build_mlp(epochs)`, an MLP with `max_iter` set to the number of epochs. For the adam optimiser, `max_iter` is the number of full passes over the training set, so each MLP in the sweep is trained for exactly that many epochs. Every model uses the same seed, so it starts from the same initial weights and sees the same shuffle order. As with the perceptron, each row is a snapshot of one training run. Stopping before the loss has converged is the point of the sweep, so scikit-learn's convergence warning is hidden.

The final model is then trained with `max_iter=2000`. scikit-learn stops early once the training loss stops improving, which happened after **1113 epochs**. Its boundary is drawn with the same `plot_boundaries` function (Figure 6).

Model (epoch)	Accuracy
MLP (1)	0.673
MLP (5)	0.677
MLP (10)	0.700
MLP (15)	0.730
MLP (20)	0.737
MLP (50)	0.767
MLP (60)	0.760
MLP (80)	0.780
MLP (100)	0.787
MLP (120)	0.807
MLP (150)	0.813
MLP (200)	0.833
MLP (1113, trained to convergence)	0.937

The final model's training accuracy is 0.927, close to its test accuracy of 0.937.

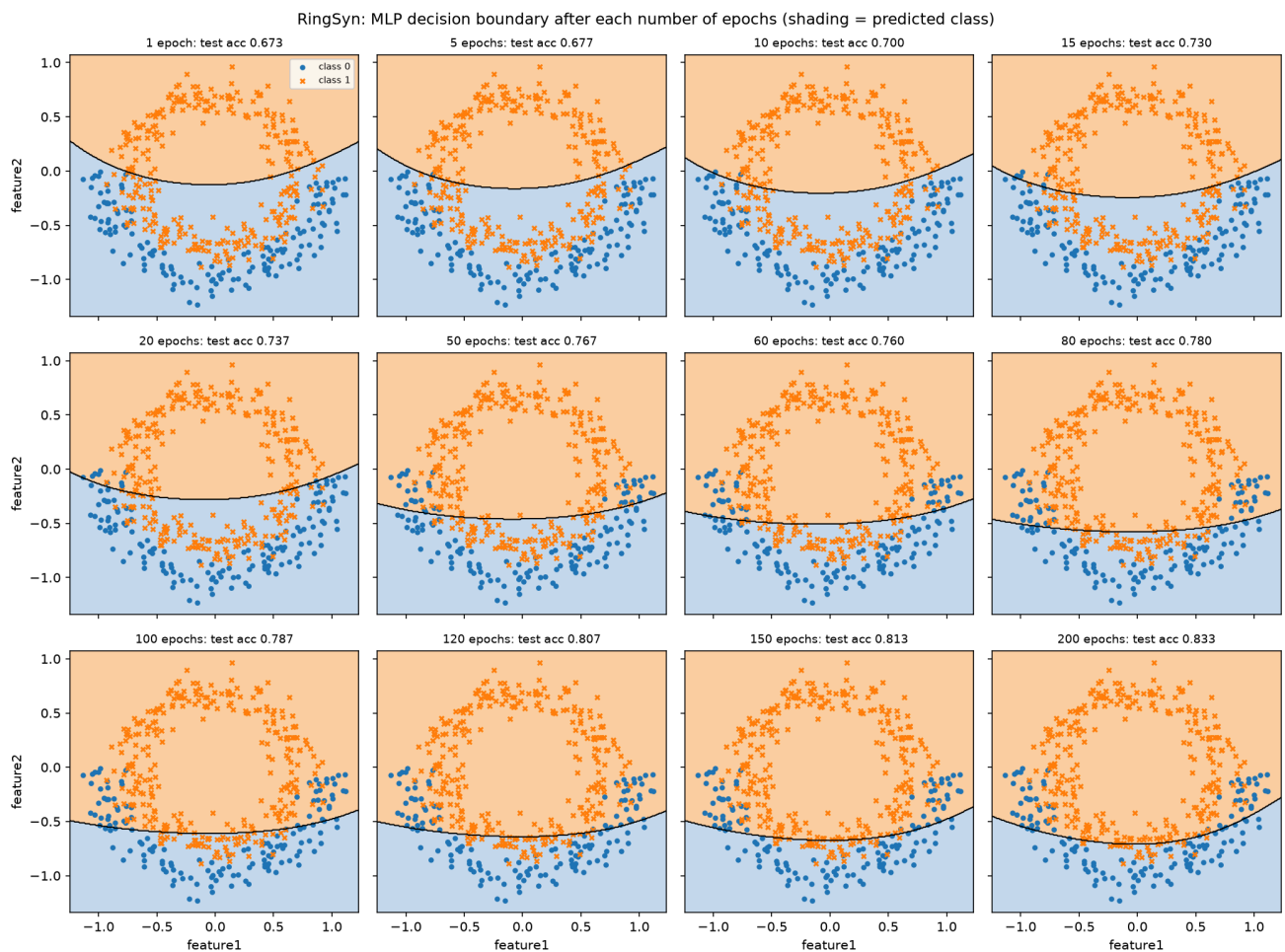


Figure 5: The MLP's decision boundary after each number of epochs, drawn over the RingSyn training data, in the same layout as Figure 4 for the perceptron.

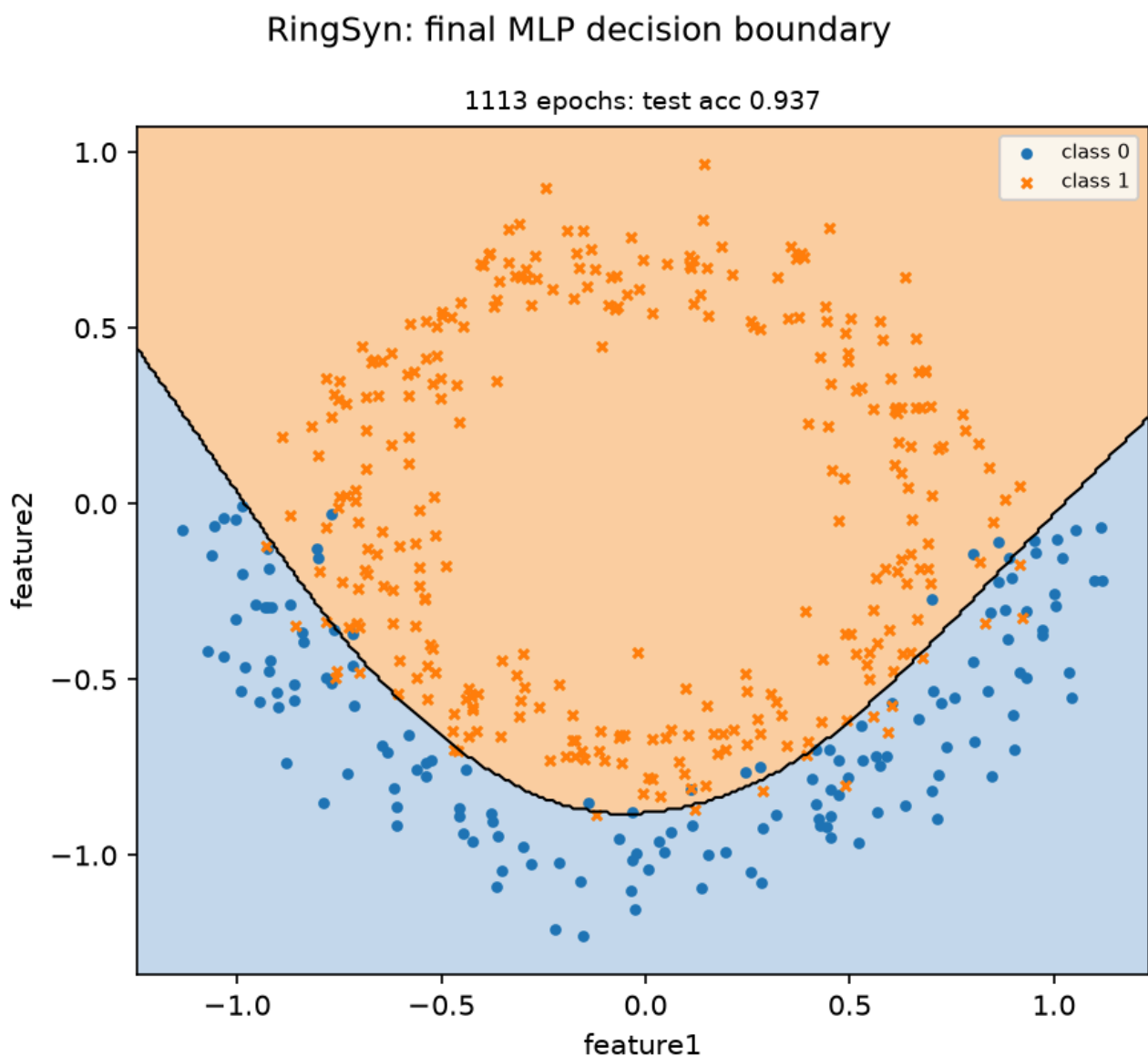


Figure 6: The decision boundary of the final MLP (1113 epochs, test accuracy 0.937), drawn over the RingSyn training data.

How does the performance of the MLP compare to the Perceptron from Task b on this dataset?

The MLP performs much better. Its test accuracy after training to convergence is **0.937**, compared with:

- 0.640–0.790 for the perceptron (0.747 after 200 epochs),
- 0.657 for the majority-class baseline.

The MLP is 19 percentage points better than the 200-epoch perceptron, 15 points better than the perceptron's best row, and 28 points better than the baseline. In terms of test mistakes, it gets 19 of the 300 test points wrong, compared with 76 for the 200-epoch perceptron.

The two models also learn very differently, as the two sweeps show:

- **The perceptron jumps around and the MLP improves steadily.** The perceptron's accuracy goes up and down from one row to the next (0.640–0.790), because each mistake moves its line by a full step. The MLP's accuracy rises in almost every row, from 0.673 after 1 epoch to 0.833 after 200, with one small dip at 60 epochs. The MLP is trained by gradient descent on a smooth loss function, and

adam takes many small steps that each reduce the loss a little. Its boundary therefore moves gradually in one direction instead of jumping around. Figure 5 shows this: the boundary starts as a shallow curve just below $\text{feature2} = 0$, slides steadily down towards the gap between the classes, and bends more as it goes.

- **The MLP needs more training.** Up to about 20 epochs, the MLP (0.673–0.737) is no better than the perceptron (0.640–0.757). After the handout's longest run of 200 epochs, it is 9 points ahead of the perceptron (0.833 vs 0.747). It only reaches 0.937 when it is allowed to train until its loss stops improving, after 1113 epochs. The MLP's advantage comes from the shape of boundary it can represent, but it takes longer to find that boundary than the perceptron takes to place a line.

Was the MLP able to better capture the structure of the data? Why or why not?

Yes. Every perceptron boundary in Figure 4 is a straight line. The final MLP's boundary in Figure 6 is a smooth U-shaped curve that wraps around the bottom of the inner ring (class 1). It follows the gap between the inner ring and the outer half-ring (class 0), and curves up on both sides between the ends of the two rings. Figure 5 shows how it got there: the boundary starts almost flat, and by 200 epochs it has started to bend into a U, with an accuracy (0.833) already higher than any perceptron row. The MLP does not need to draw a full circle. There are no class 0 points above $\text{feature2} \approx 0$, so it can leave the top of its class 1 region open. It learns the part of the structure that actually separates the two classes.

It can do this because of the hidden layer and its non-linear activation:

1. **Each hidden neuron is like a perceptron.** It computes its own weighted sum $w \cdot x + b$, so it has its own straight line.
2. **The tanh activation makes each neuron's output non-linear.** Each neuron's output changes smoothly from -1 to $+1$ as a point crosses its line, instead of growing without limit. Each neuron effectively answers "which side of my line is this point on?", with a soft transition near the line.
3. **The output neuron combines these answers.** It takes a weighted sum of the 8 hidden outputs, so it can express rules like "class 1 if the point is on the inner side of these lines on the left and of those lines on the right". Combining several soft half-planes like this gives a region whose boundary bends, and because tanh is smooth, the corners where the lines meet are rounded into a curve.

In other words, the hidden layer learns new features in which the two classes are almost linearly separable, and the output neuron then draws a linear boundary in that new space. The non-linear activation is essential. Without it, the output would be a linear function of a linear function of the input, which is still linear, and the MLP could do no better than the perceptron.

The MLP is not perfect, but its remaining errors come from the data rather than from the model. Training accuracy (0.927) is close to test accuracy (0.937), so the network is not overfitting. Most class 1 points lie between 0.5 and 0.9 from the origin, and most class 0 points lie between 0.8 and 1.2, so the two classes overlap in a thin band around 0.8–0.9. In Figure 6, the few training points on the wrong side of the boundary all sit right next to it, where the bottom of the inner ring meets the outer half-ring. Blue and orange points are mixed together there, so no smooth boundary could classify all of them correctly.

References / External resources

Part 1

- The Q1–Q5 answers were worked out by hand from the problem definition in the assignment handout. No external code was used.

- The optional checking script `part1/gantt.py` is my own code. It uses Python 3 and the Matplotlib library (J. D. Hunter, "Matplotlib: A 2D Graphics Environment", *Computing in Science & Engineering*, 9(3), 90–95, 2007) to draw Figures 1 and 2.

Part 2

- The perceptron in `part2/perceptron.ipynb` is implemented from scratch with numpy (C. R. Harris et al., "Array programming with NumPy", *Nature*, 585, 357–362, 2020), following the perceptron learning rule (F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain", *Psychological Review*, 65(6), 386–408, 1958). No machine learning library is used for the perceptron itself.
- The MLP in `part2/mlp.ipynb` uses scikit-learn's `MLPClassifier` (F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, 12, 2825–2830, 2011). It is trained with the Adam optimiser (D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization", *ICLR*, 2015).
- Figures 3–6 were drawn with Matplotlib (reference above) and saved by the notebooks to `part2/figures/`.
- The SEA concept mentioned in Task a is from W. N. Street and Y. Kim, "A Streaming Ensemble Algorithm (SEA) for Large-Scale Classification", *Proceedings of the 7th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 377–382, 2001.

Use of AI assistance (Part 2)

- I used Claude (Anthropic), an AI assistant, through Claude Code for Part 2. It wrote most of the code in `part2/perceptron.ipynb` and `part2/mlp.ipynb`, ran the notebooks, and drafted most of the Part 2 text and tables in this report from the notebook output.
- It also carried out the data checks described in the Data section and in Task a: that SeaSyn's test rows are copies of its first 40 training rows, the class balance of each dataset, the shape of the two RingSyn classes, the linear-programming test showing that SeaSyn's training data is not linearly separable, and the accuracy of the " $\text{attrib1} + \text{attrib2} > 8$ " rule.
- All results, tables and figures in Part 2 come from the saved outputs of the two notebooks, and can be reproduced by running them from top to bottom.